SQL Aggregation, Grouping & AI-Assisted Query Optimisation

AI Workshop SQL Bootcamp | Week 3 Guest Session

Instructor: Chukwuebuka Akwiwu-Uzoma

Session outline

Time	Segment	What we cover
0–5	Warm-up	Why aggregation matters and a tour of the dataset
5–17	Aggregate functions	COUNT, SUM, AVG, MIN, MAX, COUNT(DISTINCT)
17–30	GROUP BY	Grouping on one and many columns, the golden rule
30–42	Summary analytics	HAVING, FILTER, joining facts and dimensions, ROLLUP
42–57	AI for optimisation	EXPLAIN ANALYZE, prompting patterns, seven practical techniques
57–60	Wrap-up	Recap, exercises, questions

What you will be able to do by the end

- Write aggregate queries that answer real business questions.
- Use GROUP BY and HAVING correctly and explain what each is doing.
- Combine aggregation with joins to produce summary analytics.
- Read an EXPLAIN ANALYZE plan well enough to know where time is going.
- Use an AI assistant to speed up query tuning without being misled by it.

The dataset we will use today

We are working with a small e-commerce and delivery schema. Seven tables, realistic volumes, nothing exotic. Here is the shape of it:

Table	What it holds
customers	500 rows. One per customer.
orders	10,000 rows. One per order, linked to a customer.
order_details	29,727 rows. One per product line inside an order.
products	1,000 rows. Each product has a price and a department.
departments	6 rows. Electronics, Clothing, Groceries, Sports, Books, Toys.
drivers	50 rows. One per delivery driver.
delivery_trips	10,000 rows. One per trip. Status is Delivered, Pending, or Cancelled.

Every query today runs against these tables. If you want to follow along, the CSVs are in the course folder and can be loaded into Postgres with COPY.

Companion SQL files

Every code block in this handout is also available as a runnable .sql file. Open them in pgAdmin, DBeaver, or psql and run them as you follow along.

File	What it contains
01_setup.sql	CREATE TABLE statements and COPY commands to load the CSVs
02_aggregate_functions.sql	Part 2 queries: COUNT, SUM, AVG, MIN, MAX
03_group_by.sql	Part 3 queries: grouping, multi-column grouping
04_summary_analytics.sql	Part 4 queries: HAVING, FILTER, ROLLUP, joined aggregations
05_ai_optimisation.sql	Part 5 queries: EXPLAIN ANALYZE and the worked example
06_appendix_advanced_ai.sql	Appendix: pg_stat_statements, anti-patterns, advanced indexes
07_exercises.sql	Five exercises with scaffolding to get you started

Part 1 | Why aggregation matters

Raw data answers the wrong question. A table with 10,000 orders does not tell you how the business is doing. It tells you there are 10,000 orders. That is not useful on its own.

Business questions are almost always summaries:

- How many orders did we receive last month?
- Which department earns us the most revenue?
- Which drivers are cancelling too many trips?
- What is the average basket size per customer?

Aggregation is the bridge between raw rows and answers. Every aggregate query is really three decisions:

1. What do I want to measure? (the aggregate function)
2. What do I want to measure it across? (the GROUP BY)
3. Which rows should I even include? (WHERE and HAVING)

Get those three right and the rest is syntax.

Part 2 | Aggregate functions

Five functions handle 95% of what you will ever need.

Function	What it returns
COUNT(*)	Number of rows, including rows with NULL values.
COUNT(column)	Number of non-NULL values in that column.
COUNT(DISTINCT column)	Number of unique non-NULL values.
SUM(column)	Total of numeric values. Ignores NULL.
AVG(column)	Average of numeric values. Ignores NULL.
MIN(column) / MAX(column)	Smallest / largest value. Works on numbers, dates, and text.

Example 1: counting things

```
-- How many orders are in the system?
SELECT COUNT(*) AS total_orders
FROM orders;

-- How many customers have actually placed an order?
SELECT COUNT(DISTINCT customer_id) AS active_customers
FROM orders;
```

That second query is doing real work. We have 500 customers in the customers table. COUNT(DISTINCT customer_id) on orders tells us how many of them ever bought something.

Example 2: measuring value

```
-- Price spread across the catalogue
SELECT
```

```
    MIN(price)                          AS cheapest,
    MAX(price)                          AS most_expensive,
    ROUND(AVG(price)::numeric, 2)       AS average_price,
    ROUND(SUM(price)::numeric, 2)       AS total_catalog_value
FROM products;
```

Note the cast to numeric before ROUND. AVG in Postgres returns double precision for float inputs, and ROUND(double, int) does not exist. Cast to numeric and ROUND works.

Common gotchas

- COUNT(*) and COUNT(column) are not the same. If the column has nulls, the numbers differ.
- AVG ignores NULLs. That is usually what you want, but be explicit if it matters.
- Aggregate functions always return one row per group. If there is no GROUP BY, that means one row total.

Part 3 | GROUP BY

GROUP BY collapses rows that share a value into a single summary row. That is the whole idea.

Example 3: one group column

```
-- How many products does each department have?
SELECT department_id, COUNT(*) AS product_count
FROM products
GROUP BY department_id
ORDER BY product_count DESC;
```

Department IDs are numbers though, so the result is readable but not meaningful. Join to departments to fix that:

```
-- Product count and average price per department
SELECT
    d.department_name,
    COUNT(p.product_id)                 AS product_count,
    ROUND(AVG(p.price)::numeric, 2)     AS avg_price
FROM products p
JOIN departments d ON p.department_id = d.department_id
GROUP BY d.department_name
ORDER BY avg_price DESC;
```

The golden rule

Every column in your SELECT that is not inside an aggregate function must appear in GROUP BY.

If you break this rule, Postgres throws an error. That is good. In MySQL it silently returns random values, which is how bugs happen.

Example 4: grouping by more than one column

```
-- Orders per customer per month
SELECT
    customer_id,
    DATE_TRUNC('month', order_date)::date AS month,
    COUNT(*)                              AS order_count
FROM orders
GROUP BY customer_id, DATE_TRUNC('month', order_date)
ORDER BY customer_id, month;
```

DATE_TRUNC is the Postgres way of saying "round this timestamp down to the start of the month". You can also pass day, week, quarter, year.

Example 5: aggregation with WHERE

```
-- Only count orders from 2024
SELECT
    DATE_TRUNC('month', order_date)::date AS month,
    COUNT(*)                              AS orders_placed
FROM orders
WHERE order_date >= '2024-01-01'
  AND order_date <  '2025-01-01'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

WHERE runs first. It decides which rows go into the groups at all. If a row is filtered out by WHERE, it never exists as far as the aggregation is concerned.

Part 4 | HAVING and summary analytics

WHERE vs HAVING

Clause	When it runs
WHERE	Filters rows before grouping. Cannot reference aggregates.
HAVING	Filters groups after aggregation. Can reference aggregates.

Example 6: HAVING in action

```
-- Drivers who have completed more than 20 deliveries
SELECT
    driver_id,
    COUNT(*) AS completed_trips
FROM delivery_trips
WHERE status = 'Delivered'
```

```
GROUP BY driver_id
HAVING COUNT(*) > 20
ORDER BY completed_trips DESC;
```

Read it in order: WHERE keeps only delivered trips, GROUP BY buckets them by driver, HAVING throws away drivers with 20 or fewer.

Example 7: revenue by product (joins plus aggregation)

```
-- Top 10 highest-revenue products
SELECT
    p.product_name,
    SUM(od.quantity)                                AS units_sold,
    ROUND(SUM(od.quantity * p.price)::numeric, 2)   AS revenue
FROM order_details od
JOIN products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 10;
```

This is the bread and butter of analytics. Join facts (order_details) to dimensions (products), aggregate the measure (quantity times price), group by the thing you want a row per.

Example 8: the FILTER clause (Postgres specific and very clean)

```
-- Trip status breakdown per driver in a single row
SELECT
    driver_id,
    COUNT(*) FILTER (WHERE status = 'Delivered') AS delivered,
    COUNT(*) FILTER (WHERE status = 'Pending')   AS pending,
    COUNT(*) FILTER (WHERE status = 'Cancelled') AS cancelled,
    COUNT(*)                                     AS total_trips,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE status = 'Cancelled') / COUNT(*),
        2
    ) AS cancellation_rate_pct
FROM delivery_trips
GROUP BY driver_id
ORDER BY cancellation_rate_pct DESC;
```

FILTER is cleaner than SUM(CASE WHEN ... THEN 1 END) and does the same job. If you are coming from MySQL or SQL Server, take note. This is one of the nicest things about Postgres.

Example 9: ROLLUP for sub-totals and grand totals

```
-- Orders per department per month, with department totals and a grand total
SELECT
```

```
    d.department_name,
    DATE_TRUNC('month', o.order_date)::date AS month,
    COUNT(DISTINCT o.order_id)              AS orders,
    SUM(od.quantity)                        AS units
FROM orders o
JOIN order_details od ON o.order_id  = od.order_id
JOIN products p       ON od.product_id = p.product_id
JOIN departments d    ON p.department_id = d.department_id
GROUP BY ROLLUP (d.department_name, DATE_TRUNC('month', o.order_date))
ORDER BY d.department_name NULLS LAST, month NULLS LAST;
```

ROLLUP adds a summary row at each level. NULL in the grouping column means "this is the sub-total or grand total row". Useful when you want an executive summary and the detail in one query.

Part 5 | How AI can support query optimisation

Here is the honest version. AI is very good at reading query plans and suggesting hypotheses. It is not good at knowing what your data actually looks like. Use it for the first, verify the second yourself.

Start with the real tool: EXPLAIN ANALYZE

Before you go near an AI, run EXPLAIN ANALYZE. It executes the query and tells you exactly what Postgres did.

```
EXPLAIN ANALYZE
SELECT
    c.name,
    COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY order_count DESC;
```

Three things to look for in the output:

- Seq Scan on a large table. Means Postgres read every row. Sometimes fine, often a sign a missing index.
- Rows Removed by Filter is much higher than rows returned. Means you read a lot to throw most of it away.
- Actual time at each node. The node with the biggest time is where your query is spending its life.

Where AI genuinely helps

1. Reading the plan. Paste EXPLAIN ANALYZE output into your AI assistant and ask for the bottlenecks. It is very good at this.
2. Suggesting indexes. Give it the query and schema, ask what indexes would help.
3. Rewriting queries. A correlated subquery can often be flattened into a join. AI spots these patterns fast.
4. Explaining plan nodes. HashAggregate, Nested Loop, Materialize, BitmapHeapScan. AI will tell you what each one is doing and why it was chosen.

A prompt template that actually works

```
I have this Postgres query:

[paste query]

The table schemas are:

[paste CREATE TABLE statements, including existing indexes]
```

```
The EXPLAIN (ANALYZE, BUFFERS) output is:

[paste output]

The query currently runs in X seconds against ~N rows.
I need it under Y seconds.

What are the top 3 changes I should make, ranked by expected impact?
For each change, explain what it does and what the risk is.
```

The shape matters. Give the AI the query, the schema, the plan, and the target. Without all four it is guessing.

Worked example

Say this query is slow:

```
SELECT c.name, SUM(od.quantity * p.price) AS lifetime_spend
FROM customers c
JOIN orders o        ON o.customer_id = c.customer_id
JOIN order_details od ON od.order_id  = o.order_id
JOIN products p      ON p.product_id  = od.product_id
WHERE o.order_date >= '2024-01-01'
GROUP BY c.name
ORDER BY lifetime_spend DESC
LIMIT 20;
```

You run EXPLAIN ANALYZE. You see a Seq Scan on orders with a filter on order_date. You paste the plan into Claude and ask what to do. Almost certainly it suggests:

```
CREATE INDEX idx_orders_order_date ON orders (order_date);
-- or a composite one if order_date and customer_id both matter:
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);
```

You add the index. You re-run EXPLAIN ANALYZE. The Seq Scan becomes an Index Scan. The runtime drops. You now know the suggestion was right, because you verified it.

Where AI will mislead you

- It suggests indexes that already exist. Always run \d tablename first and paste real schema, not assumed schema.
- It cannot see your data distribution. An index is useless if 90% of the rows match your filter.
- It happily recommends adding five indexes on a table that gets a thousand inserts a minute. Every index has a write cost. AI does not know your write pattern.
- It guesses at Postgres version features. If you are on 12, it will suggest something from 15. Check the docs.

- It can hallucinate functions and syntax. Always try the suggestion in a safe environment first.

The workflow to actually use

1. Write the query.
2. Run EXPLAIN (ANALYZE, BUFFERS) on it.
3. Give the AI the query, schema, indexes, and plan.
4. Apply one suggestion at a time.
5. Re-run EXPLAIN ANALYZE. If it did not help, revert and try the next.
6. Keep the winners. Document why.

Last rule. AI gives you hypotheses. The database gives you answers. Never skip step 5.

Appendix | Going deeper with AI-assisted optimisation

The main session covered the basics. This appendix is reference material you can work through after class. These are the techniques that move you from "my query is faster now" to "I actually understand my database".

A1. Finding what to optimise in the first place

Most of the time, the slow query is not the one you are looking at. It is the one you run a thousand times a day without noticing. pg_stat_statements is the extension that tracks this.

```
-- Enable once (requires superuser + restart)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Top 10 queries by total time spent
SELECT
    substring(query, 1, 80)             AS query_preview,
    calls,
    ROUND(total_exec_time::numeric, 0)  AS total_ms,
    ROUND(mean_exec_time::numeric, 2)   AS mean_ms,
    rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Paste the top 10 into an AI assistant and ask which of these look like they have the most room to improve. It will spot repeated patterns like N+1 queries and obvious missing indexes across the workload, not just in one query.

A2. Anti-patterns AI spots instantly

These are the rewrites AI will suggest nine times out of ten. Worth learning to spot them yourself.

Functions on indexed columns

```
-- SLOW: function on the column disables the index
SELECT * FROM orders
WHERE DATE(order_date) = '2024-01-15';

-- FAST: same meaning, index usable
SELECT * FROM orders
WHERE order_date >= '2024-01-15'
  AND order_date <  '2024-01-16';
```

Leading wildcards in LIKE

```
-- SLOW: cannot use a btree index
```

```
SELECT * FROM products
WHERE product_name LIKE '%phone%';

-- FAST: trigram index + ILIKE
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_products_name_trgm
    ON products USING gin (product_name gin_trgm_ops);
-- now this query uses the index
SELECT * FROM products
WHERE product_name ILIKE '%phone%';
```

NOT IN with nullable columns

```
-- SUBTLE BUG: if any row in the subquery has a NULL customer_id,
-- this returns zero rows. Even when you expect results.
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);

-- SAFE: NOT EXISTS handles NULLs correctly and is usually faster
SELECT c.* FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

SELECT * when you only need a few columns

SELECT * forces Postgres to read the whole row. If you only need two columns and they are both in an index, an index-only scan can skip the table entirely. Always list columns explicitly in production code.

OR across different columns

```
-- SLOW: OR prevents using indexes on either column
SELECT * FROM orders
WHERE customer_id = 42 OR order_date = '2024-06-01';

-- FAST: UNION lets each side use its own index
SELECT * FROM orders WHERE customer_id = 42
UNION
SELECT * FROM orders WHERE order_date = '2024-06-01';
```

A3. Index strategies AI will suggest

A plain single-column index is not the end of the conversation. These four patterns cover most real tuning work.

Composite indexes (order matters)

```
-- Supports queries filtering on customer_id alone,
-- OR customer_id AND order_date.
-- Does NOT efficiently support queries filtering only on order_date.
CREATE INDEX idx_orders_customer_date
    ON orders (customer_id, order_date);
```

Rule: put the most selective and most-often-filtered column first. AI will generally get this right if you describe your query patterns.

Partial indexes

```
-- Only index the rows we actually query.
-- Tiny index, huge speedup for the specific query.
CREATE INDEX idx_pending_trips
    ON delivery_trips (driver_id)
    WHERE status = 'Pending';

-- This query now uses a very small index
SELECT * FROM delivery_trips
WHERE status = 'Pending' AND driver_id = 12;
```

If 2% of rows match your filter, a partial index is 50x smaller and faster than a full one.

Covering indexes (INCLUDE)

```
-- Enables index-only scans: Postgres answers the query
-- without touching the table at all.
CREATE INDEX idx_orders_customer_covering
    ON orders (customer_id) INCLUDE (order_date, order_id);

-- This can now run as an index-only scan
SELECT order_id, order_date
FROM orders
WHERE customer_id = 100;
```

Expression indexes

```
-- If you always query by month, index by month
CREATE INDEX idx_orders_month
    ON orders (DATE_TRUNC('month', order_date));

-- Uses the index
SELECT COUNT(*) FROM orders
WHERE DATE_TRUNC('month', order_date) = '2024-06-01';
```

A4. Materialised views for aggregation-heavy workloads

If you are running the same GROUP BY query hourly and it takes 30 seconds, stop. Materialise it.

```
-- Pre-compute the expensive aggregation
CREATE MATERIALIZED VIEW mv_daily_revenue AS
SELECT
    DATE(o.order_date) AS day,
    d.department_name,
    SUM(od.quantity * p.price) AS revenue
FROM orders o
JOIN order_details od ON o.order_id  = od.order_id
JOIN products p       ON od.product_id = p.product_id
JOIN departments d    ON p.department_id = d.department_id
GROUP BY DATE(o.order_date), d.department_name;

-- Index the view just like a table
CREATE INDEX idx_mv_daily_revenue_day
    ON mv_daily_revenue (day);

-- Refresh on a schedule (CONCURRENTLY keeps it readable during refresh)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue;
```

Trade-off: the data is as fresh as the last refresh. For dashboards and reports, that is usually fine. For live operational queries, it is not.

A5. Making AI suggestions actually safe

Before you apply anything an AI suggests, run through this checklist.

- Does the index already exist? Run \d tablename and check. AI cannot see this.
- What is the write pattern on this table? Each index slows inserts and updates. If the table gets 10k writes per minute, be conservative.
- What is the data distribution? Run SELECT status, COUNT(*) FROM delivery_trips GROUP BY status. An index on a column where 99% of values are the same is often useless.
- Are statistics fresh? Run ANALYZE tablename. Postgres planner uses stats to choose plans. Stale stats produce bad plans and bad AI suggestions.
- Are you on the Postgres version the suggestion assumes? INCLUDE needs 11+. Merge-append needs 12+. Check before you debug a syntax error.
- Did it actually help? Re-run EXPLAIN ANALYZE. If the runtime and buffer reads did not drop, revert.

A6. Using AI for schema reviews

Paste your CREATE TABLE statements and ask: "What are the three biggest risks in this schema?" AI is very good at catching:

- Missing foreign keys (orders.customer_id with no FK constraint).

- Columns that should be NOT NULL but are not (order_date, status).
- Wrong data types (TEXT for dates, INTEGER for phone numbers, VARCHAR(255) as a default for everything).
- Obvious normalisation problems (department_name stored on every product row).
- Missing indexes on columns that are clearly foreign keys or frequently filtered.

A five-minute schema review with AI catches problems that would take hours to debug in production.

A7. The prompts that work better than "make this faster"

Prompt quality determines output quality. These specific framings produce much better answers than vague asks.

Instead of	Try
Make this query faster.	Here is my query, schema, existing indexes, and EXPLAIN ANALYZE output. What are the top 3 bottlenecks, ranked by expected impact?
What index should I add?	My workload is 80% reads, 20% writes. Query patterns are X, Y, Z. What indexes would help, and which would hurt more than they help?
Why is this slow?	This query takes 4 seconds on 10M rows. The plan shows a Seq Scan on orders. Explain what the planner is doing and why it chose this over an index scan.
Rewrite this query.	Here is a query with three correlated subqueries. Rewrite it to use joins, preserve the exact same semantics including NULL handling, and explain each change.

The pattern: give context, state the goal, and ask for reasoning. AI is much better at this than at one-word tuning requests.

Seven more practical ways AI helps with query work

These are the techniques I reach for most often in real data engineering work. Each one has a prompt pattern that tends to produce good output.

1. Spotting anti-patterns before they bite you

AI is excellent at reading a query and pointing out things that will hurt performance or correctness. Common ones it catches:

- SELECT * pulling columns you do not need (more I/O, more network).
- Functions wrapping an indexed column in WHERE, like WHERE DATE(order_date) = '2024-01-01'. The index becomes unusable.
- OR across different columns preventing index use. Can often be rewritten as UNION ALL.
- Unnecessary DISTINCT hiding a broken join that is producing duplicates.
- Implicit CROSS JOIN from a forgotten ON clause.

- LIKE '%something%' which cannot use a standard B-tree index (consider pg_trgm).

Prompt template:

```
Review this Postgres query for anti-patterns and performance issues.
For each one, tell me:
 - what the problem is
 - why it hurts performance
 - the rewrite

[paste query]
```

2. Picking the right index type

"Add an index" is a blunt suggestion. Postgres has several types and they are not interchangeable.

Type	When to use
B-tree	Default. Equality and range on scalar columns. 90% of cases.
GIN	JSONB, arrays, full-text search, pg_trgm for LIKE '%text%' queries.
BRIN	Very large tables where data is naturally ordered (time-series, logs). Tiny size, coarse lookups.
Partial	Indexed rows are a small subset. E.g. WHERE status = 'Pending' when 95% are Delivered.
Covering (INCLUDE)	Query reads only indexed columns, so Postgres skips the table entirely.

Example of a partial index that pays off if most delivery trips are already Delivered:

```
CREATE INDEX idx_trips_pending
ON delivery_trips (order_id)
WHERE status = 'Pending';
```

Ask AI: "Given this query and table, what type of index is best and why?" It is good at this specifically because the reasoning is pattern-based.

3. Suggesting materialized views for expensive aggregations

If a query is slow, runs often, and does not need to be real-time, a materialized view is the cleanest fix. It runs the aggregation once, stores the result like a table, and you read from it.

```
-- Expensive query we run on every dashboard load
CREATE MATERIALIZED VIEW daily_department_revenue AS
SELECT
    d.department_name,
    DATE_TRUNC('day', o.order_date)::date         AS day,
```

```
    SUM(od.quantity * p.price)::numeric(14, 2)    AS revenue,
    COUNT(DISTINCT o.order_id)                    AS orders
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN products p       ON od.product_id = p.product_id
JOIN departments d    ON p.department_id = d.department_id
GROUP BY d.department_name, DATE_TRUNC('day', o.order_date);

-- Add an index on the common filter column
CREATE INDEX idx_dept_rev_day ON daily_department_revenue (day);

-- Refresh on a schedule (cron, Airflow, pg_cron)
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_department_revenue;
```

Ask AI: "Is this query a good candidate for a materialized view? What is the refresh strategy?" The second half of that question is where AI adds real value.

4. Translating queries across SQL dialects

You inherit a query in MySQL or SQL Server and need it running on Postgres by Friday. AI is very good at this specific task because the mapping is largely syntactic.

What to watch for after translation:

- Date functions (GETDATE vs NOW, DATEADD vs interval arithmetic).
- String concatenation (+ vs || vs CONCAT).
- TOP N vs LIMIT N.
- NULL handling in aggregations (slight differences between engines).
- Identifier quoting (square brackets vs double quotes vs backticks).

Always run EXPLAIN ANALYZE on the translation. Same SQL does not mean same plan.

5. Generating benchmark harnesses

When AI says "this should be faster", do not believe it. Measure it. Ask AI to write the benchmark script too.

```
-- Prompt: "Write a pgbench custom script that runs both versions
-- of this query 500 times and captures latency percentiles."

-- Or a simpler in-SQL version:
DO $$
DECLARE
    start_time timestamptz;
    elapsed_ms numeric;
    i int;
BEGIN
    start_time := clock_timestamp();
    FOR i IN 1..100 LOOP
```

```
        PERFORM COUNT(*) FROM orders WHERE order_date >= '2024-01-01';
    END LOOP;
    elapsed_ms := EXTRACT(EPOCH FROM clock_timestamp() - start_time) * 1000;
    RAISE NOTICE 'Version A: % ms total, % ms avg', elapsed_ms, elapsed_ms / 100;
END $$;
```

6. Finding the slow queries in production

You do not know which query is hurting you until you measure. The pg_stat_statements extension tracks every query and its stats.

```
-- One-time setup (needs superuser)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Find your worst offenders
SELECT
    substring(query, 1, 80)                AS query_preview,
    calls,
    ROUND(mean_exec_time::numeric, 2)      AS avg_ms,
    ROUND(total_exec_time::numeric, 2)     AS total_ms,
    ROUND((100.0 * total_exec_time /
           SUM(total_exec_time) OVER ())::numeric, 2) AS pct_of_total
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

Paste those 20 rows into an AI and ask which to attack first. Total time matters more than average time: a query that runs a million times at 5ms is a bigger problem than one that runs once at 5 seconds.

7. Schema review before it becomes a problem

The cheapest optimisation is the one you never need, because the schema was right to start with. AI is a fast second opinion on schema design.

Good prompts:

- "Review these CREATE TABLE statements. Are the data types appropriate for the likely values?"
- "Based on these typical access patterns, what indexes would you recommend?"
- "Which of these columns should probably be NOT NULL?"
- "Would partitioning help this table? By what key?"

Common things AI catches in a review: TEXT where VARCHAR with a cap is fine, NUMERIC where INT would do, missing NOT NULL on columns that always have values, missing indexes on foreign keys.

One more principle worth saying out loud

AI assistants are very confident. They will give you an answer even when they do not know. Treat their suggestions like a helpful junior engineer: often right, sometimes wrong, always needs verification. Your

EXPLAIN ANALYZE and your pg_stat_statements are the ground truth. The AI is a very fast second pair of eyes.

Wrap-up and exercises

Quick recap

- Aggregate functions collapse many rows into one summary value.
- GROUP BY decides what that summary is a summary of.
- WHERE filters rows. HAVING filters groups. They run at different stages.
- FILTER, ROLLUP, and DATE_TRUNC are Postgres friends. Use them.
- For optimisation: read the plan first, ask the AI second, verify always.

Exercises to try on your own

Try these against the course dataset. Answers in your next session.

1. For each department, show the total revenue generated in 2024 (quantity × price across all orders in 2024).
2. Find drivers whose cancellation rate is above 10% and who have made at least 100 trips.
3. For the top 5 customers by lifetime spend, show total number of orders and average order value.
4. Find any day where total orders were more than double the previous day.
5. Pick one of your queries. Run EXPLAIN ANALYZE. Paste the output into an AI tool. Write down the first suggestion it gave and whether it actually helped.

Further reading

- PostgreSQL documentation on Aggregate Functions and GROUP BY.
- "Use The Index, Luke!" (use-the-index-luke.com). Free, short, and the best intro to indexing you will find.
- The Postgres EXPLAIN documentation. Dry but essential.